

Computer Architecture ISA and Pipeline Review

Ting-Jung Chang

NYCU CS

Course Administration

- Course video uploaded
- Lab1 released. Due two weeks later.
- PS1 released

Agenda

- ISA Review
- Pipeline Review

Architecture vs. Microarchitecture

- **Architecture/Instruction Set Architecture:**
 - Programmer visible state (memory and register)
 - Operations (Instructions and how they work)
 - Execution Semantics (Interrupts)
 - Input/Output
 - Data Types/Sizes
- **Microarchitecture/Organization:**
 - Tradeoffs on how to implement ISA for some metric (speed, energy, cost)
 - Examples: pipeline depth, number of pipelines, cache size, silicon area, peak power, execution ordering, bus widths, ALU widths

Software Developments

up to 1955

Libraries of numerical routines

- Floating point operations
- Transcendental functions
- Matrix manipulation, equation solvers

1955-60

High level languages – Fortran 1956

Operating Systems

- Assemblers, loaders, linkers, compilers
- Accounting programs to keep track of usage and charges

Machines required **experienced** operators

- Most users could not be expected to understand these programs, much less write them
- Machines had to be sold with a lot of resident software

Compatibility Problem at IBM

- By early 1960's, IBM had 4 incompatible lines of computers!
 - 701 → 7094
 - 650 → 7074
 - 702 → 7080
 - 1401 → 7010
- Each system had its own
 - Instruction set
 - I/O system and secondary storage (magnetic tapes, drums and disks)
 - Assemblers, compilers, libraries
 - Market niche business, scientific, real time, ...

IBM 360!

IBM 360: A General-Purpose Register (GPR) Machine

- Processor State
 - 16 general-purpose 32-bit registers
 - May be used as index and base register
 - Register 0 has some special properties
 - 4 floating point 64-bit registers
 - A program status word (PSW)
 - PC, condition codes, control flags
- A 32-bit machine with 24-bit addresses
 - But no instruction contains a 24-bit address!
- Data formats
 - 8-bit bytes, 16-bit half words, 32-bit words, 64-bit double words

The IBM 360 is why bytes are 8-bits long today!

IBM 360: Initial Implementations

	Model 30	Model 70
Storage	8K – 64 KB	256K – 512 KB
Datapath	8-bit	64-bit
Circuit Delay	30 nsec/level	5 nsec/level
Local Store	Main Store	Transistor Registers
Control Store	Read only 1μsec	Conventional circuits

- IBM 360 instruction set architecture (ISA) completely hid the underlying technological differences between various models.
- Milestone: The first true ISA designed as portable hardware-software interface!

With minor modifications it still survives today!

IBM 360: Over 50 years later...

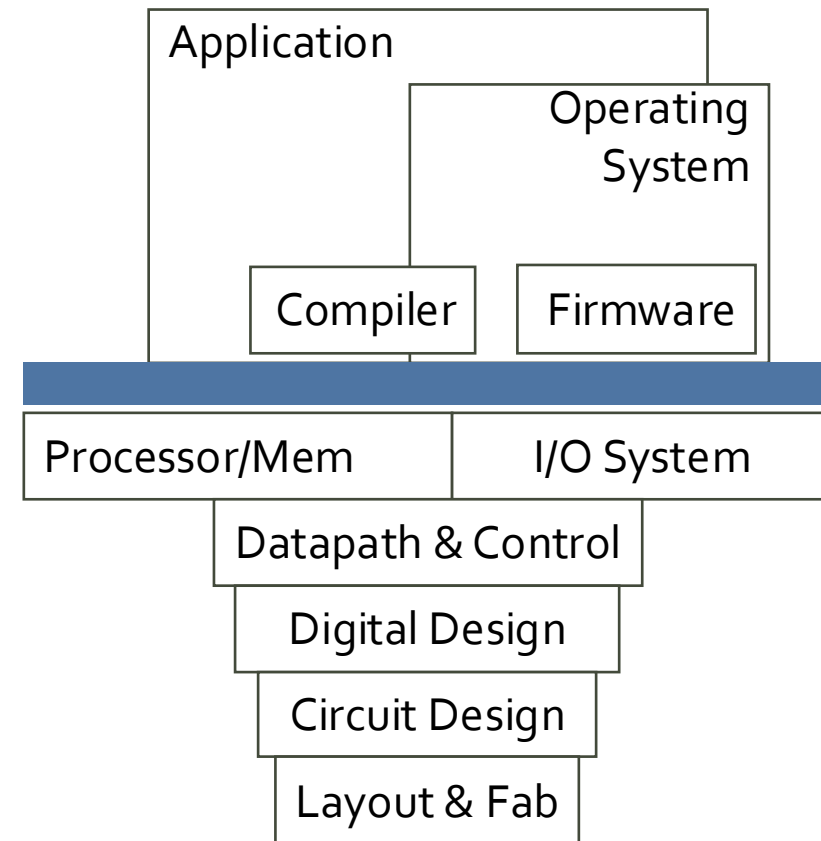
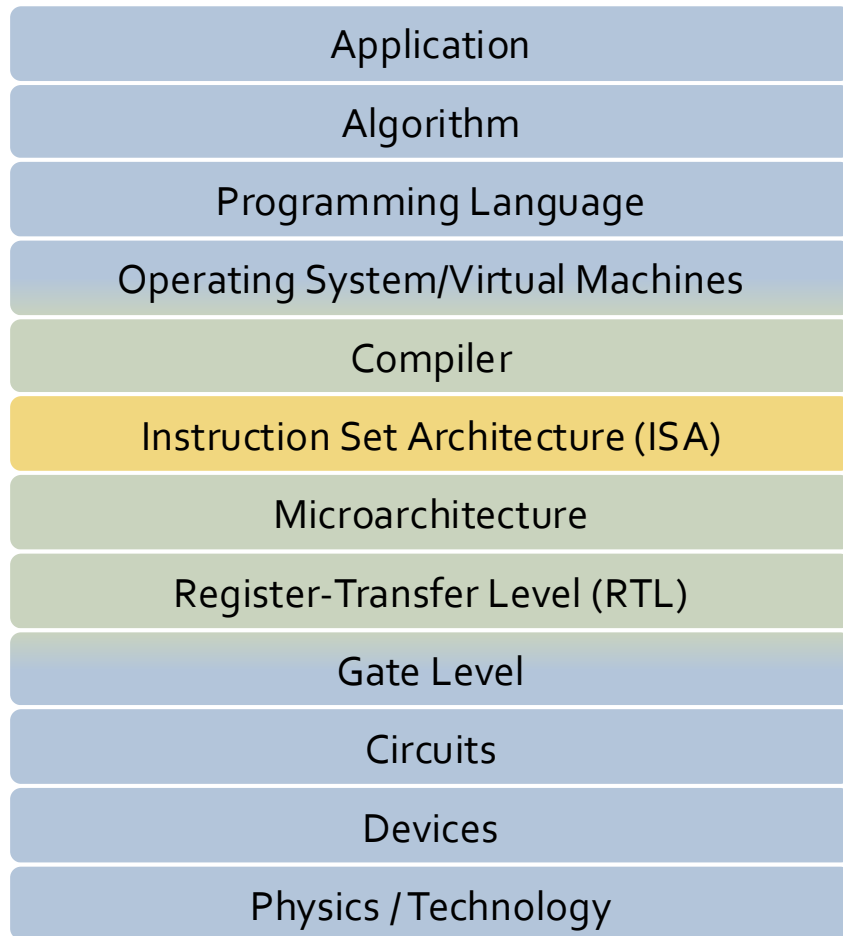
The zSeries z16 Microprocessor



Image Credit: IBM
© International Business
Machines Corporation.

- 5.2GHz in 7nm Samsung technology
- 22.5 Billion transistors in 530 mm²
- 64-bit virtual addressing – original S/360 was 24-bit, and S/370 was 31-bit extension
- 8 cores + L2s per chip
- 128 KB L1, 32 MB L2, 256MB L3, 2048 MB L4
- Integrated on-chip AI accelerator

Instruction Set Architecture (ISA)

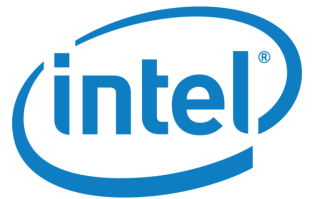


Instruction Set Architecture (ISA)

- The ISA is functional abstraction of the processor (a “mental model”)
 - What operations can be performed
 - How to name storage locations
 - The format (bit pattern) of the instructions
- ISA typically does NOT define
 - Timing of the operations
 - Power used by operations
 - How operations/storage are implemented
- Many implementations possible for a given ISA

Different Instruction Set Architecture

- ARM
 - Family of ISAs developed by ARM
 - Widely used in mobile and low-power devices
 - Now expanding into desktop, datacenters and cloud servers
- x86
 - Family of ISAs developed by Intel (and AMD)
 - Used in general-purpose computing systems (desktops and servers)
- RISC-V
 - Open standard ISAs developed at UC-Berkeley
 - Mostly used in embedded systems
 - Early adoption in AI accelerators and research chips



Same Architecture/Different Microarchitecture

Intel Xeon

- x86 instruction set
- 32+ cores
- 200W+
- Decode 6 instructions/cycle/core
- Large shared caches (tens of MB L3)
- Out-of-order
- ~ 2.0-3.5 GHz

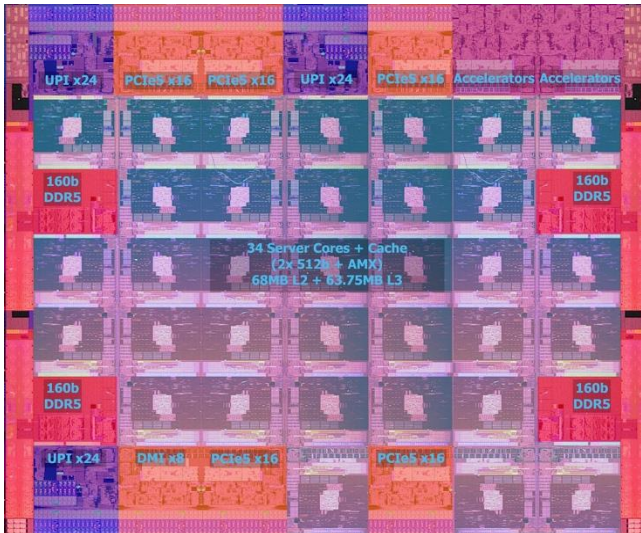


Image Credit: Intel

Intel Atom

- x86 instruction set
- Few cores (1–8)
- 2W
- Decode 2 instructions/cycle/core
- tens of KB L1, <2 MB L2
- In-order
- 1.6GHz

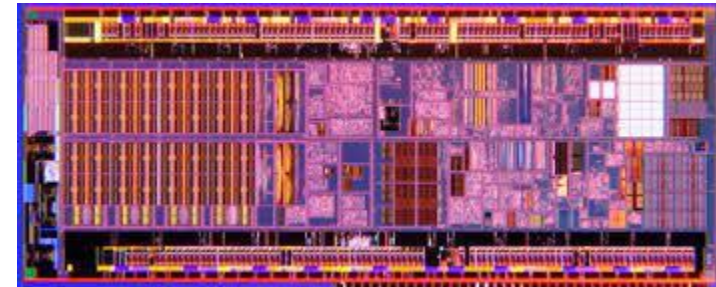


Image Credit: Intel

Different Architecture/Different Microarchitecture

Intel Xeon

- x86 instruction set
- 32+ cores
- 200W+
- Decode 6 instructions/cycle/core
- Large shared caches (tens of MB L3)
- Out-of-order
- ~ 2.0-3.5 GHz

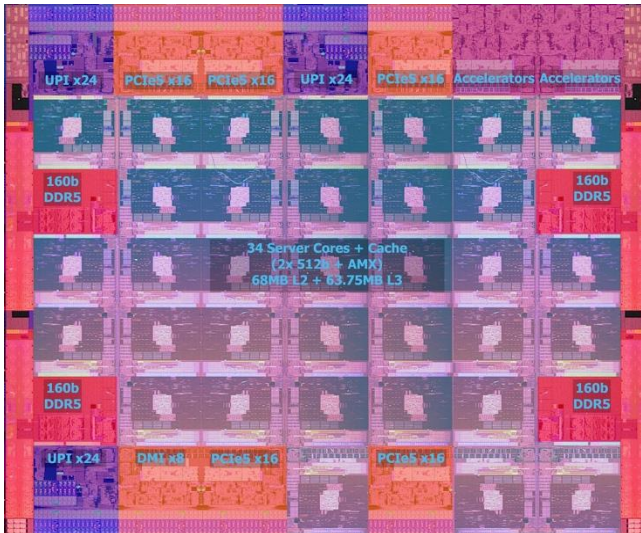


Image Credit: Intel

Apple M2 Ultra

- ARM instruction set
- Up to 24 cores
- ~90–100 W
- Decode 8+ instructions/cycle/core
- Large shared caches across clusters
- Out-of-order
- ~3.5 GHz peak frequency

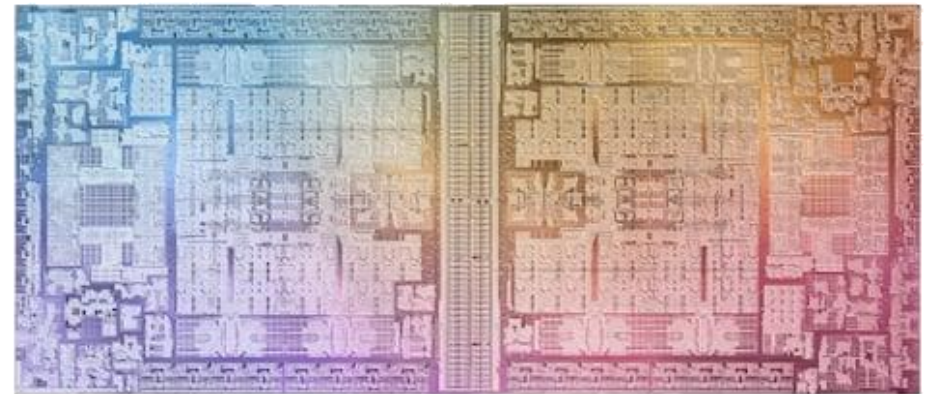


Image Credit: Apple

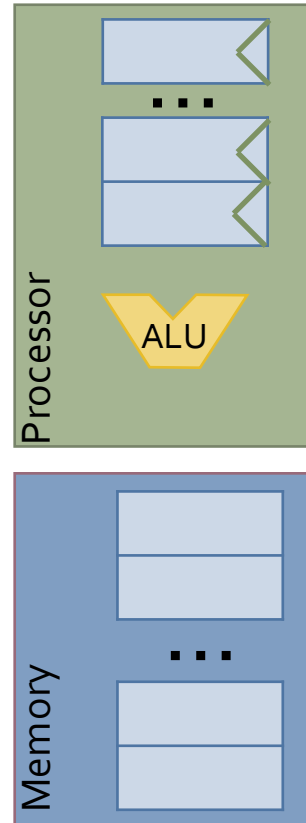
Key ISA Decisions

- Operands
 - How many?
 - Location?
 - Addressing mode?
 - Types?
- Operations
 - What kind?
 - How many?
- Instruction format
 - Length(s) of bit pattern
 - Which bits mean what

ISA Classification

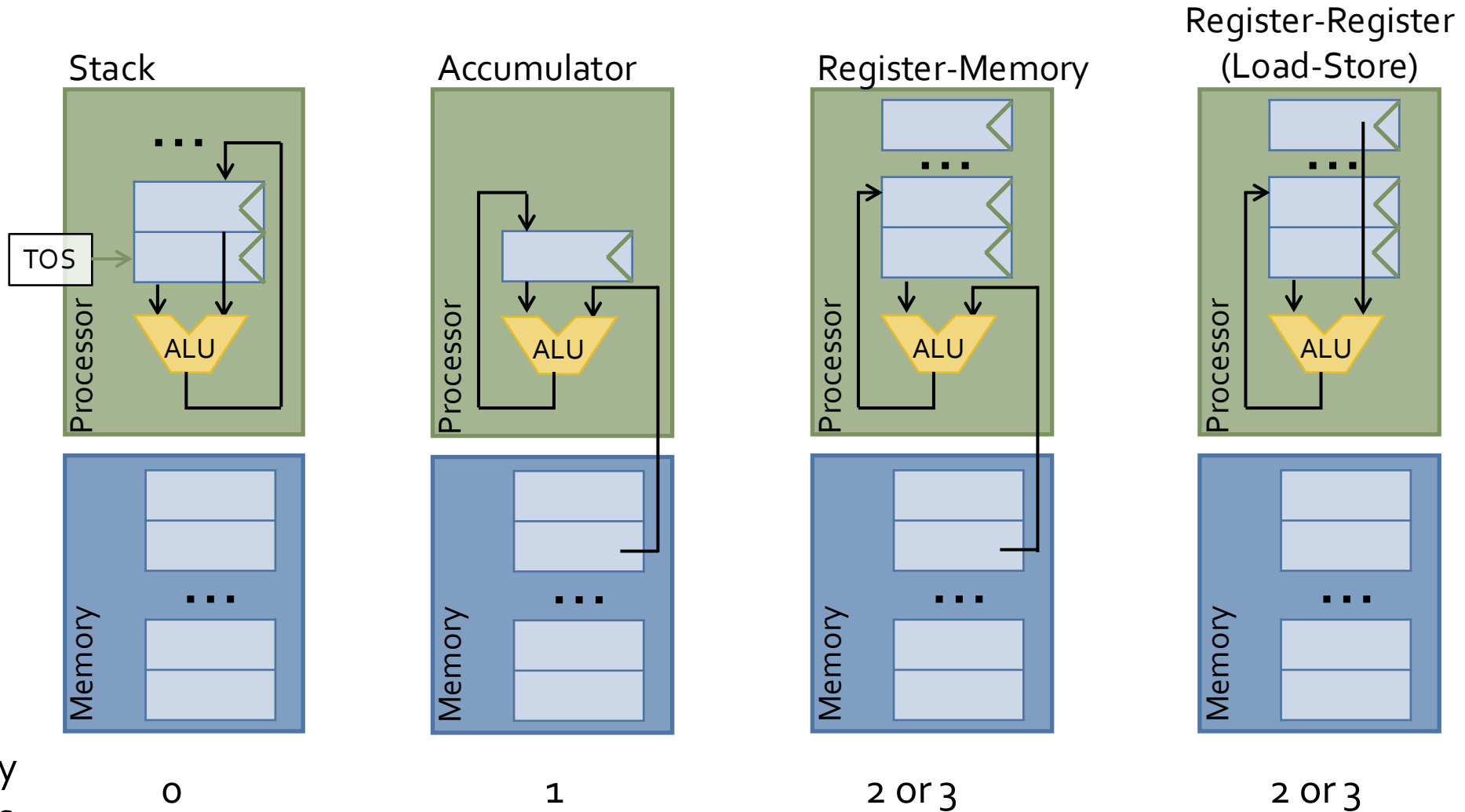
Operands

- Operands may be from
 - Stack
 - Accumulator
 - Register
 - Register and Memory



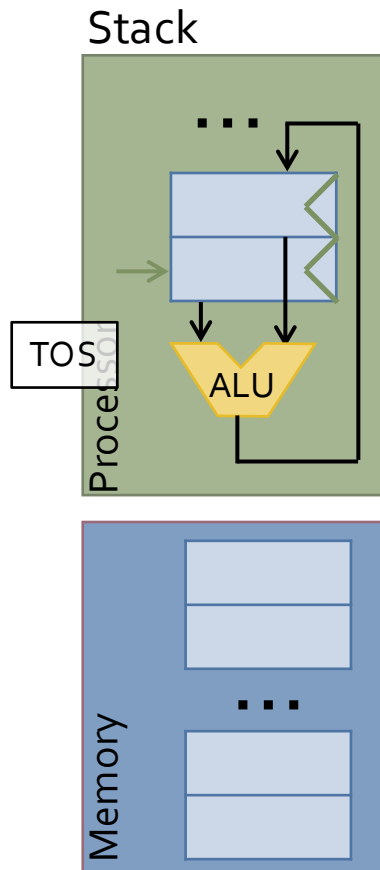
Where do operands come from
and where do results go?

Machine Models of ISA

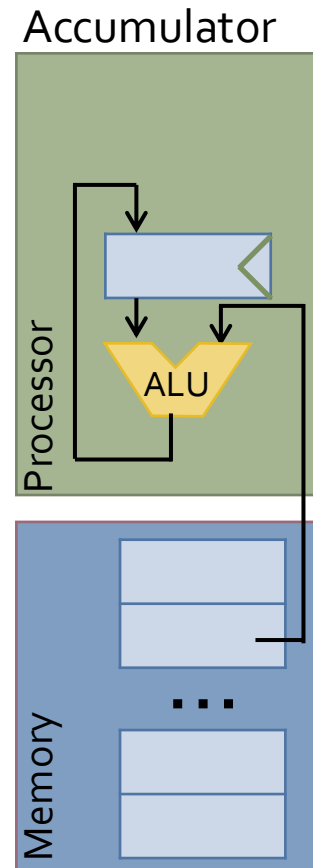


Summary: Machine Model

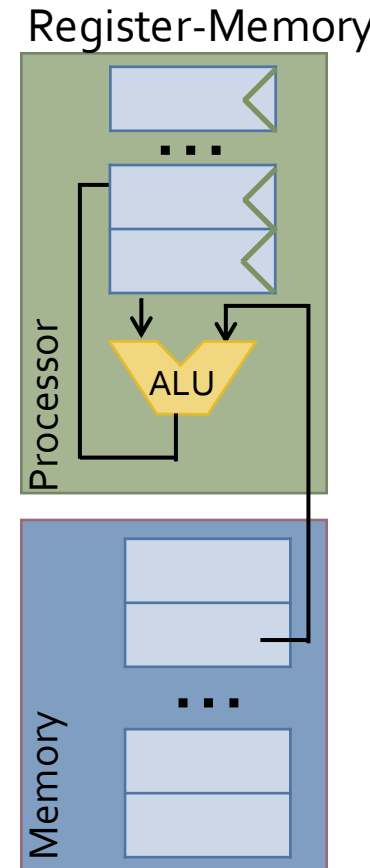
$$C = A + B$$



Push A
Push B
Add
Pop C

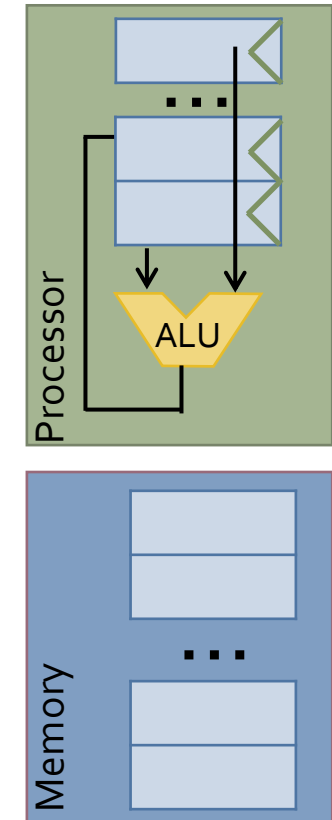


Load A
Add [B]
Store C



Load R1, [A]
Add R3, R1, [B]
Store R3, [C]

Register-Register
(Load-Store)



Load R1, [A]
Load R2, [B]
Add R3, R1, R2
Store R3, [C]

ISA Classification

Operand Addressing Mode

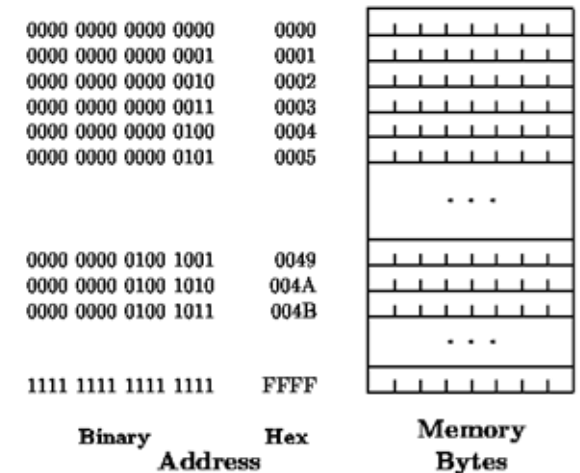
- Addressing mode specifies how operands are located from memory and/or registers.
- Common addressing modes
 - Register
 - Immediate
 - Register indirect
 - Displacement
 - Indexed
 - Direct
 - Memory indirect
 - Auto-increment and auto-decrement
 - Scaled

Memory Addressing

- Objects have byte addresses
 - the number of bytes counted from the beginning of memory

- Object Length:

- bytes (8 bits), half words (16 bits)
- words (32 bits), and double words (64 bits).
- The type is implied in opcode, e.g.,
 - lb – load byte
 - lw – load word, etc.



- Addressing mode specifies how operands are located from memory and/or registers

Memory Addressing

- Byte Ordering

- Little Endian: Stores the least significant byte (LSB) at the lowest memory address

- Ex: 0x12345678

Address	0x100	0x101	0x102	0x103
	0x78	0x56	0x34	0x12

- Big Endian: Stores the most significant byte (MSB) at the lowest memory address

- Ex: 0x12345678

Address	0x100	0x101	0x102	0x103
	0x12	0x34	0x56	0x78

- Problem occurs when exchanging data among machines with different orderings

Summary: Addressing Modes

Addressing Mode	Instruction	Function
Register	Add R ₄ , R ₃ , R ₂	Regs[R ₄] <- Regs[R ₃] + Regs[R ₂] **
Immediate	Add R ₄ , R ₃ , #5	Regs[R ₄] <- Regs[R ₃] + 5 **
Displacement	Add R ₄ , R ₃ , 100(R ₁)	Regs[R ₄] <- Regs[R ₃] + Mem[100 + Regs[R ₁]]
Register Indirect	Add R ₄ , R ₃ , (R ₁)	Regs[R ₄] <- Regs[R ₃] + Mem[Regs[R ₁]]
Absolute/Direct	Add R ₄ , R ₃ , (0x475)	Regs[R ₄] <- Regs[R ₃] + Mem[0x475]
Memory Indirect	Add R ₄ , R ₃ , @(R ₁)	Regs[R ₄] <- Regs[R ₃] + Mem[Mem[Regs[R ₁]]]
PC relative	Add R ₄ , R ₃ , 100(PC)	Regs[R ₄] <- Regs[R ₃] + Mem[100 + PC]
Scaled	Add R ₄ , R ₃ , 100(R ₁)[R ₅]	Regs[R ₄] <- Regs[R ₃] + Mem[100 + Regs[R ₁] + Regs[R ₅] * 4]

** May not actually access memory!

Type And Size of Operands

- Types
 - Binary Integer
 - Binary Coded Decimal (BCD)
 - Floating Point
 - IEEE 754
 - Cray Floating Point
 - Intel Extended Precision (80-bit)
 - Packed Vector Data
 - Addresses
- Width
 - Binary Integer (8-bit, 16-bit, 32-bit, 64-bit)
 - Floating Point (32-bit, 40-bit, 64-bit, 80-bit)
 - Addresses (16-bit, 24-bit, 32-bit, 48-bit, 64-bit)

Key ISA Decisions

- Operands
 - How many?
 - Location?
 - Addressing mode?
 - Types?
- Operations
 - What kind?
 - How many?
- Instruction format
 - Length(s) of bit pattern
 - Which bits mean what

Classes of Instructions

- Arithmetic & Logical
 - Integer arithmetic: add, sub, mul, div, shift
 - Logical operation: and, or, xor, not
- Data Transfer
 - copy, load, store (w/ memory addressing)
- Control Flow
 - branch, jump, call, return, trap
- Floating Point
- System: OS and memory management
- Graphics: pixel and vertex, compression/decompression operations
- String
 - move, compare, search

Key ISA Decisions

- Operands
 - How many?
 - Location?
 - Addressing mode?
 - Types?
- Operations
 - What kind?
 - How many?
- Instruction format
 - Length(s) of bit pattern
 - Which bits mean what

Encoding an Instruction Set

- Opcode: specifying the operation
- # of operands:
 - addressing mode
 - address specifier tells what addressing mode is used
 - Load-store computer
 - Only one memory operand
 - Only one or two addressing modes
- The architecture must balance several competing forces when encoding the instruction set:
 - # of registers & addressing modes
 - Size of registers & addressing mode fields
 - Average instruction size & average program size.
 - Easy to handle in pipeline implementation

Example: x86 Instruction Encoding

Instruction Prefixes	Opcode	ModR/M	Scale, Index, Base	Displacement	Immediate
Up to four Prefixes (1 byte each)	1,2, or 3 bytes	1 byte (if needed)	1 byte (if needed)	0,1,2, or 4 bytes	0,1,2, or 4 bytes

Possible instructions 1 to 15 bytes long

RISC-V Instruction Encoding

- Restrictions
 - 4 bytes per instruction
 - Different instructions have different parameters (registers, immediates, ...)
 - Various fields should be encoded to consistent locations
 - Simpler decoding circuitry
- RISC-V uses 6 “types” of instruction encoding

Name (Field Size)	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	Comments
R-type	funct7	rs2	rs1	funct3	rd	opcode	Arithmetic instruction format
I-type	immediate[11:0]		rs1	funct3	rd	opcode	Loads & immediate arithmetic
S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode	Stores
SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode	Conditional branch format
UJ-type	immediate[20,10:1,11,19:12]				rd	opcode	Unconditional jump format
U-type	immediate[31:12]				rd	opcode	Upper immediate format

ISA Classification

Instruction Length

- **Fixed Width:** Every Instruction has same width
 - + Easy to decode (**RISC** Architectures: MIPS, PowerPC, SPARC, ARM, RISC-V...)
 - Wasted bits in instructions (Why is this bad?)
 - Harder-to-extend ISA (how to add new instructions?)
 - Ex: MIPS, RISC-V, ARM, ...
- **Variable Length:** Instructions can vary in width
 - + Takes less space in memory and caches (**CISC** Architectures: x86, VAX...)
 - More logic to decode a single instruction
 - Harder to decode multiple instructions concurrently
 - Ex: x86

ISA Classification

Instruction Length

- **Hybrid**
 - Support 16-bit and 32-bit instructions in RISC. Narrow instructions support fewer operations, smaller address and immediate fields, fewer registers, and two-address format rather than the classic three-address format
 - Claim a code size reduction of up to 40%
 - Ex: ARM Thumb, MIPS16e, and RISC-V
- **Compressed:**
 - PowerPC and some VLIWs (Store instructions compressed, decompress into Instruction Cache)
- **(Very) Long Instruction Word (VLIW):**
 - Multiple instructions in a fixed width bundle
 - Ex: Multiflow, HP/ST Lx, TI C6000

Instruction Length Tradeoffs

- Instructions are eventually encoded with 0s and 1s.
 - Each instruction is encoded into several bytes of binary numbers.
- Tradeoffs
 - Code size (memory space, bandwidth, latency) vs. hardware complexity
 - ISA extensibility and expressiveness
 - Performance? Smaller code vs. imperfect decode

Real World Instruction Sets

RISC? CISC?

Arch	Type	# Oper	# Mem	Data Size	# Regs	Addr Size	Use
Alpha	Reg-Reg	3	0	64-bit	32	64-bit	Workstation
ARM	Reg-Reg	3	0	32/64-bit	16	32/64-bit	Cell Phones, Embedded
MIPS	Reg-Reg	3	0	32/64-bit	32	32/64-bit	Workstation, Embedded
SPARC	Reg-Reg	3	0	32/64-bit	24-32	32/64-bit	Workstation
TI C6000	Reg-Reg	3	0	32-bit	32	32-bit	DSP
IBM 360	Reg-Mem	2	1	8/16/32/64-bit	16	24/31/64-bit	Mainframe
x86	Reg-Mem	2	1	8/16/32/64-bit	4/8/16	16/32/64-bit	Personal Computers, HPC
RISC-V	Reg-Reg	3	0	32/64/128-bit	32	32/64-bit	Embedded

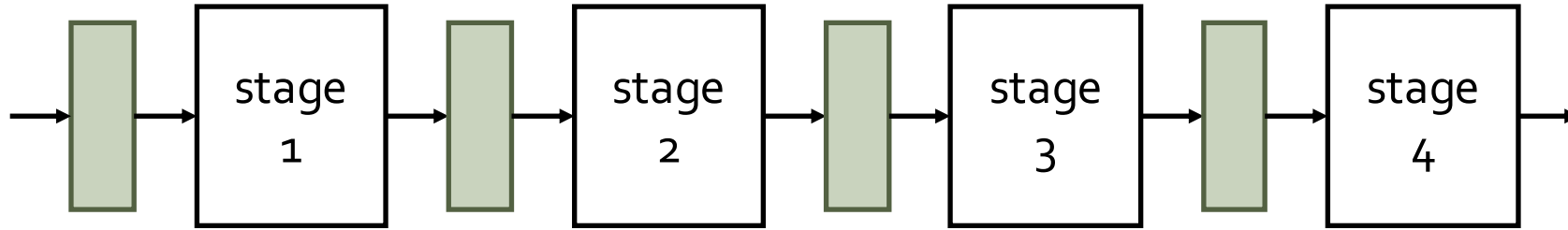
Why the Diversity in ISAs?

- Technology influenced ISA
 - Storage is expensive, tight encoding space
 - Reduced Instruction Set Computer
 - Remove instructions until whole computer fits on die
 - Multicore/Manycore
- Application Influenced ISA
 - Instructions for applications
 - DSP instructions
 - Compiler technology has improved

Agenda

- ISA Review
- Pipeline Review
 - Pipelining Basics
 - Hazards
 - Structural Hazards
 - Data Hazards
 - Control Hazards

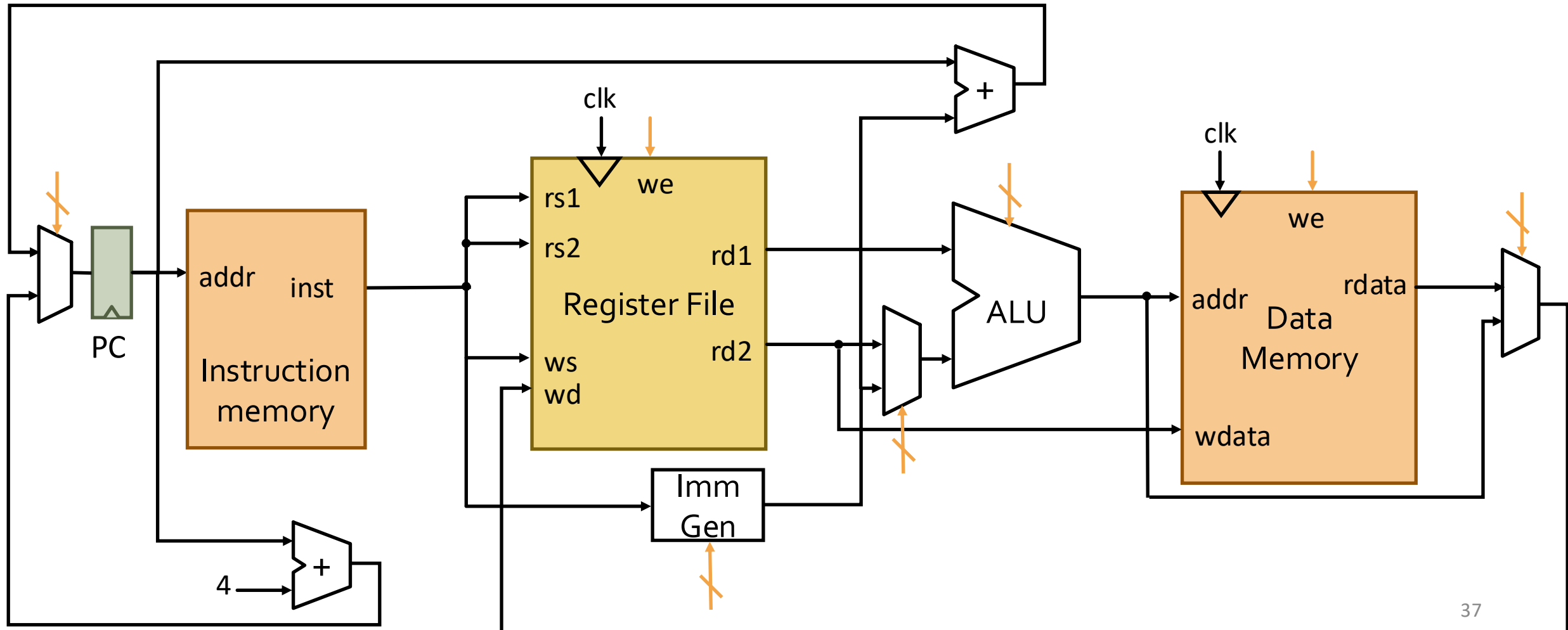
An Ideal Pipeline



- All objects go through the same stages
- No sharing of resources between any two stages
- Propagation delay through all pipeline stages is equal
- The scheduling of an object entering the pipeline is not affected by the objects in other stages

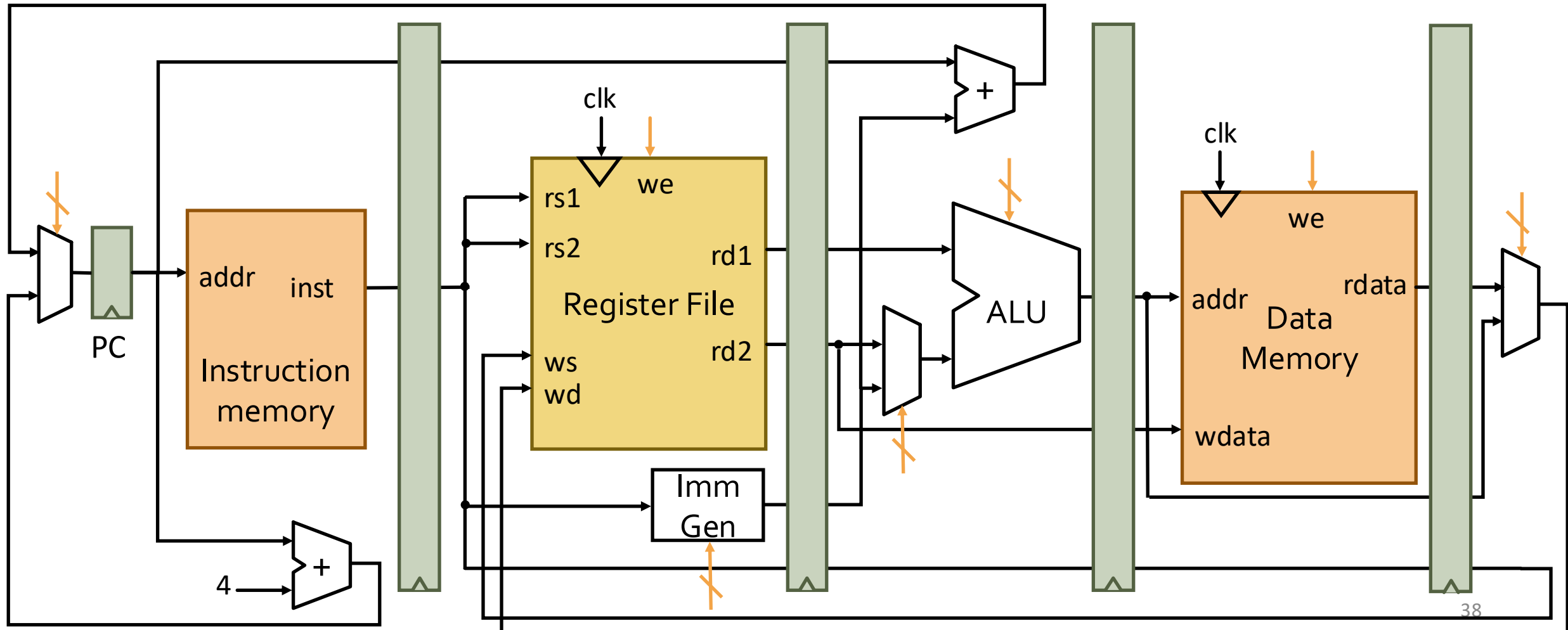
These conditions generally hold for industrial assembly lines,
but instructions depend on each other!

Single Cycle Datapath

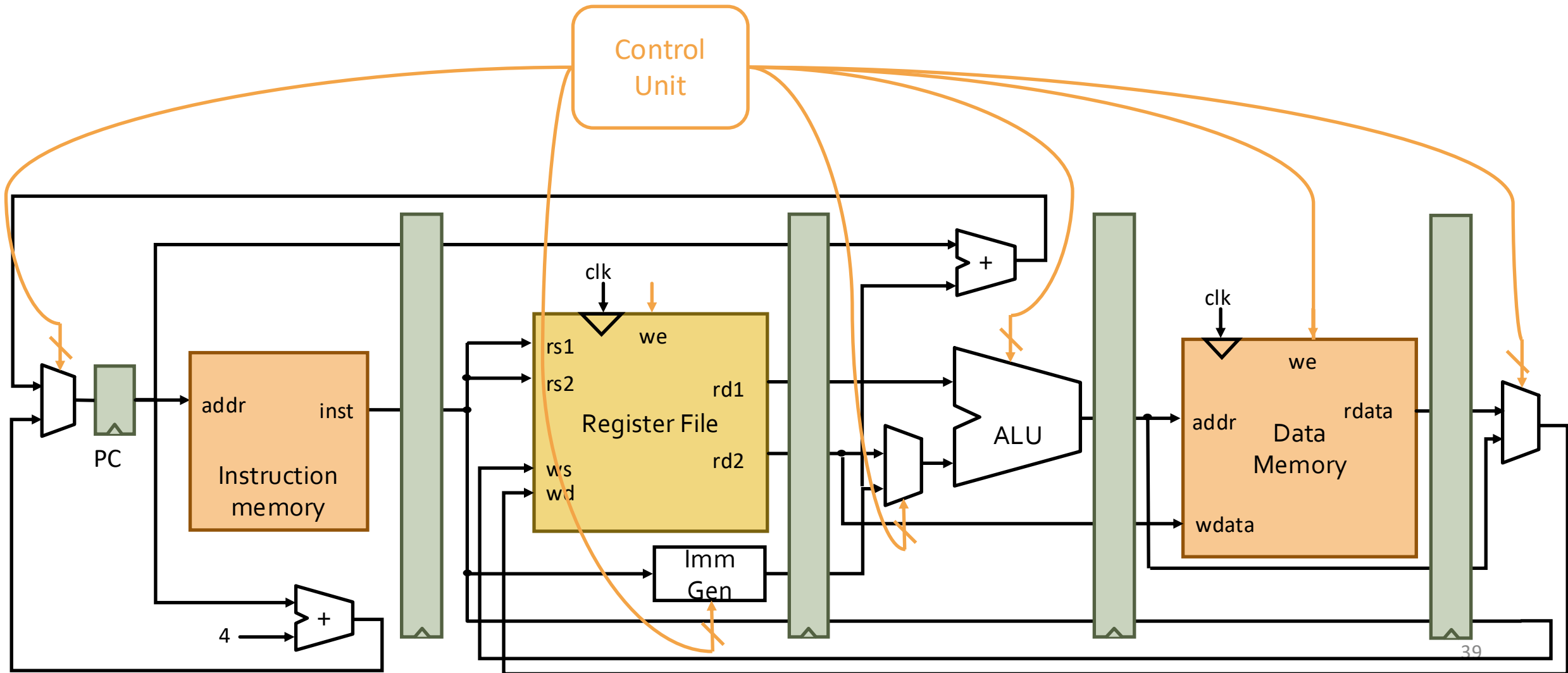


Pipelined Datapath

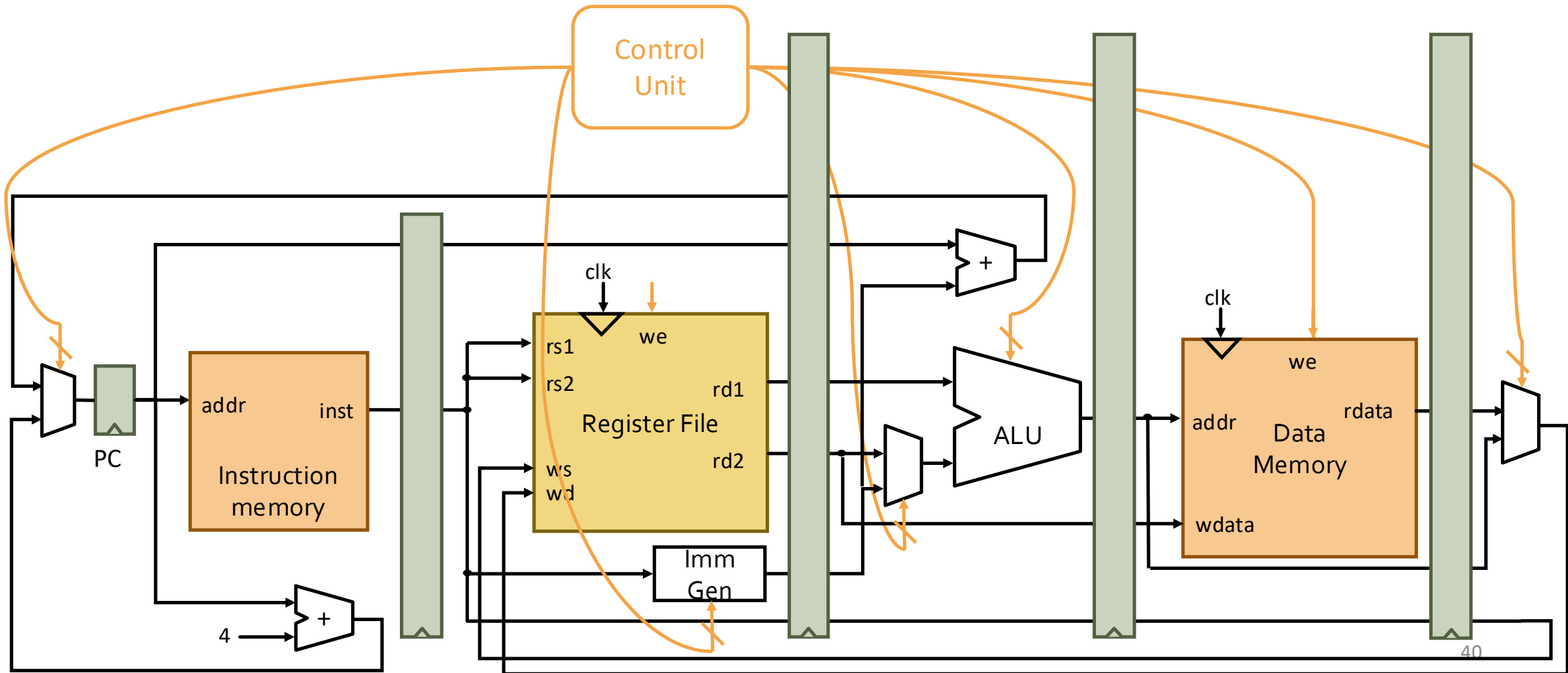
Clock period can be reduced by dividing the execution of an instruction into multiple cycles
 $t_C > \max \{t_{IM}, t_{RF}, t_{ALU}, t_{DM}, t_{RW}\}$ (= t_{DM} **probably**)



Pipelined Control



Pipelined Control



Iron Law of Processor Performance

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} * \frac{\text{Cycles}}{\text{Instruction}} * \frac{\text{Time}}{\text{Cycle}}$$

- Instructions per program depends on source code, compiler technology, and ISA
- Cycles per instructions (CPI) depends on ISA and μ architecture
- Time per cycle depends upon the μ architecture and base technology

Microarchitecture	CPI	cycle time
Single-cycle unpipelined	1	long
Multi-cycle, unpipelined control	>1	short
Pipelined	1	short

CPI Examples

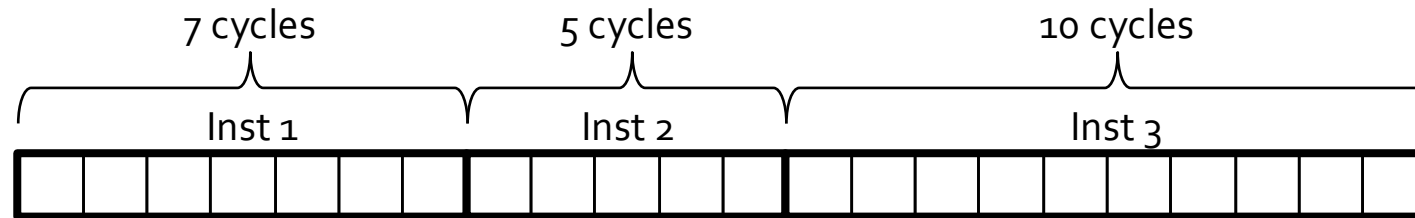
Time $\longrightarrow$

Single Cycle
machine



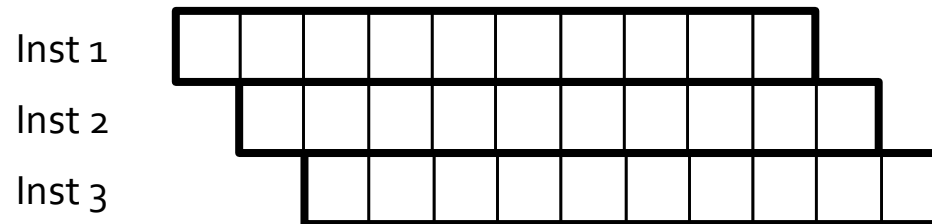
3 instructions
3 cycles
 $CPI=1$

Multi Cycle
machine



3 instructions
22 cycles
 $CPI=7.33$

Pipelined
machine



3 instructions
3 cycles
 $CPI=1$

Instructions Interact With Each Other in Pipeline

- **Structural Hazard:** An instruction in the pipeline may need a resource being used by another instruction in the pipeline
- **Data Hazard:** An instruction depends on a data value produced by an earlier instruction
- **Control Hazard:** Whether an instruction should be executed depends on a control decision made by an earlier instruction

Agenda

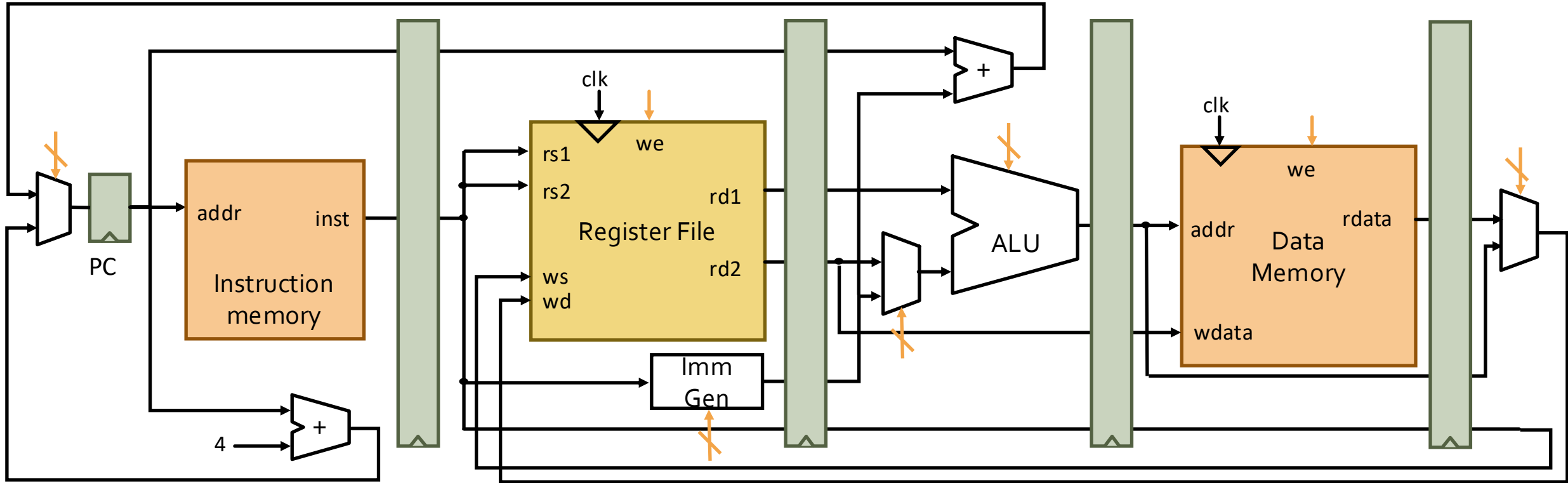
- ISA Review
- Pipeline Review
 - Pipelining Basics
 - Hazards
 - Structural Hazards
 - Data Hazards
 - Control Hazards

Overview of Structural Hazards

- Structural hazard occurs when two instructions need same hardware resource at same time
- Approaches to resolving structural hazards
 - **Schedule:** Programmer explicitly avoids scheduling instructions that would create structural hazards
 - **Stall:** Hardware includes control logic that stalls until earlier instruction is no longer using contended resources
 - **Duplicate:** Add more hardware to design so that each instruction can access independent resources at the same time
- Our 5-stage pipeline has no structural hazards by design

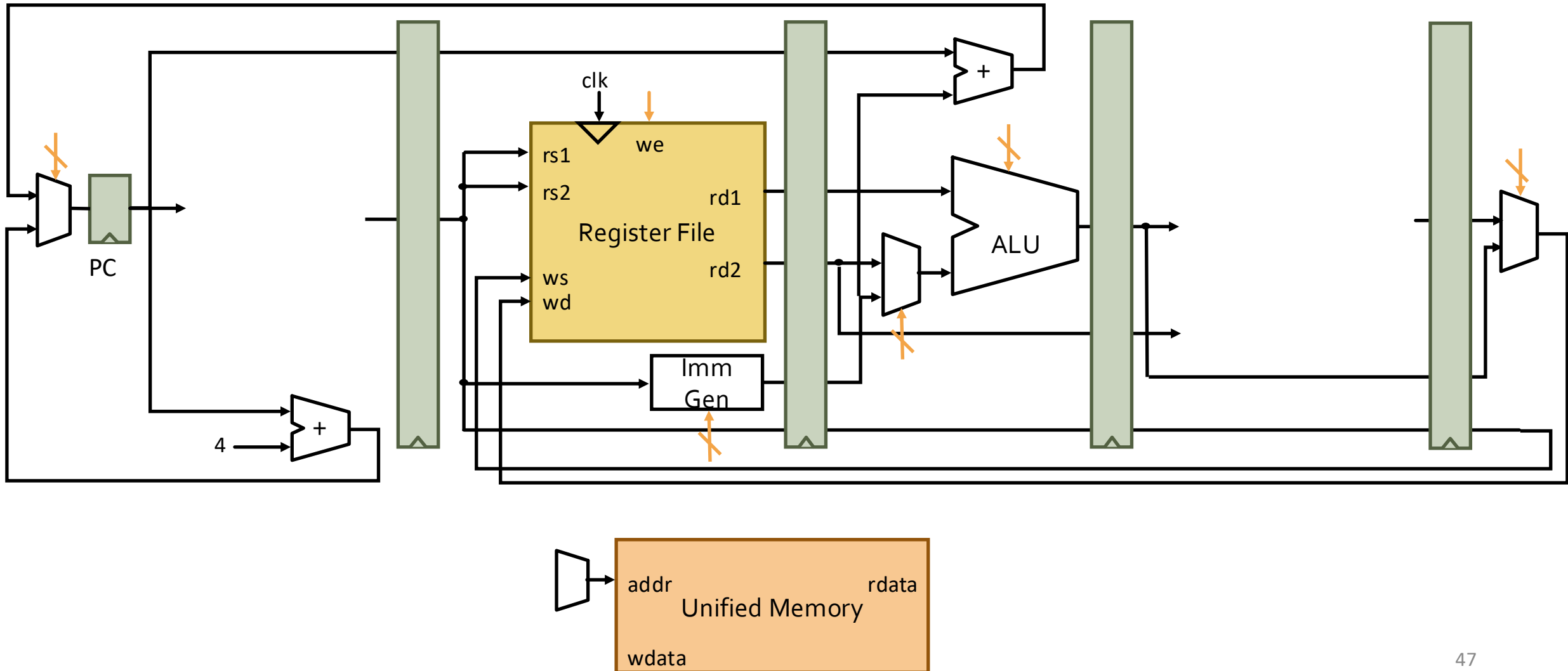
Example Structural Hazard

Unified Memory



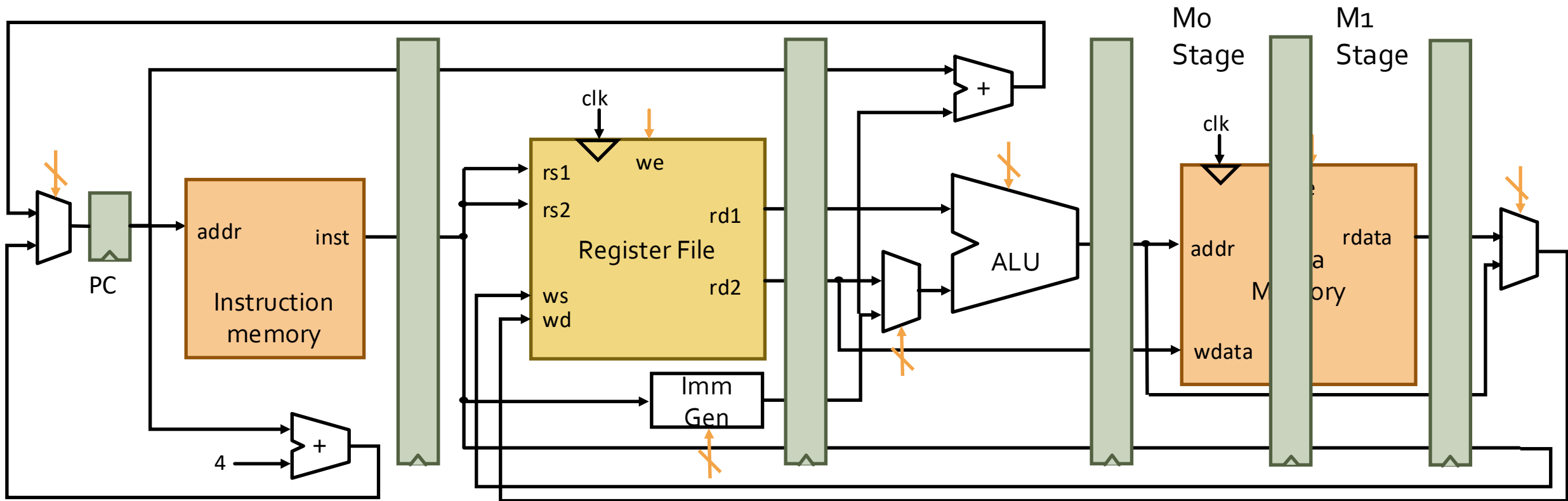
Example Structural Hazard

Unified Memory



Example Structural Hazard

2-Cycle Memory



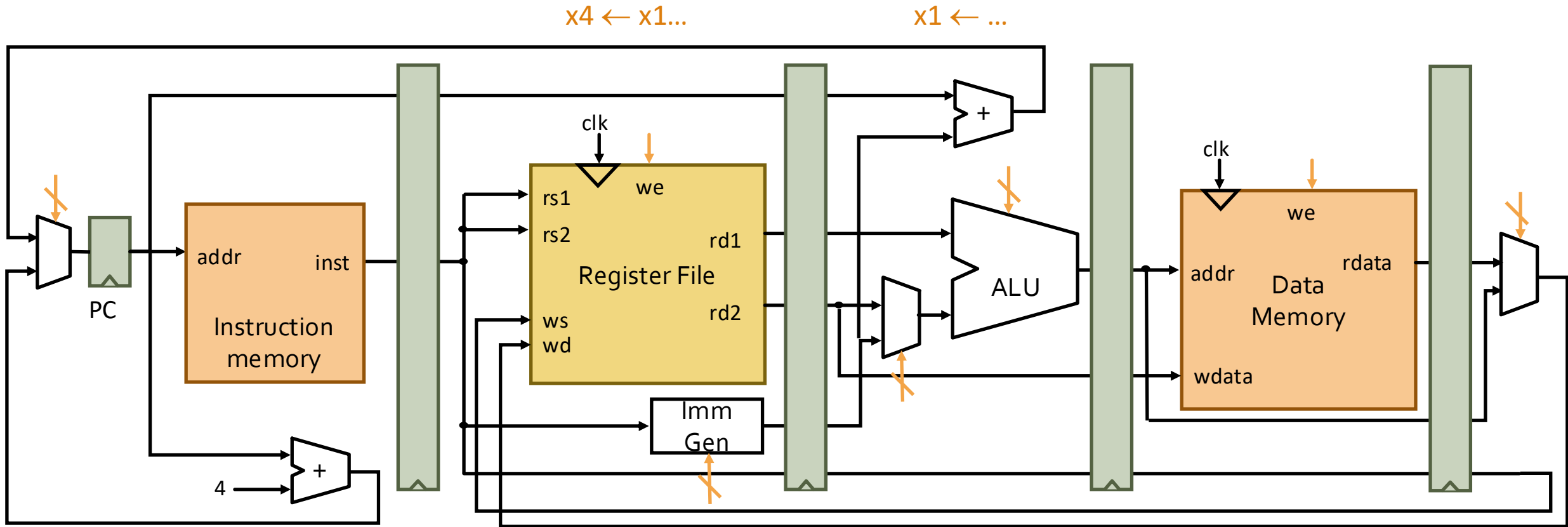
Agenda

- ISA Review
- Pipeline Review
 - Pipelining Basics
 - Hazards
 - Structural Hazards
 - Data Hazards
 - Control Hazards

Overview of Data Hazards

- Data hazard occurs when one instruction depends on a data value produced by a preceding instruction still in the pipeline
- Approaches to resolving data hazards
 - **Schedule:** Programmer explicitly avoids scheduling instructions that would create data hazards
 - **Stall:** Hardware includes control logic that freezes earlier stages until preceding instruction has finished producing data value
 - **Bypass:** Hardware datapath allows values to be sent to an earlier stage before preceding instruction has left the pipeline
 - **Speculate:** Guess that there is not a problem, if incorrect kill speculative instruction and restart

Example Data Hazard



...

$x1 \leftarrow x0 + 10$

$x4 \leftarrow x1 + 17$

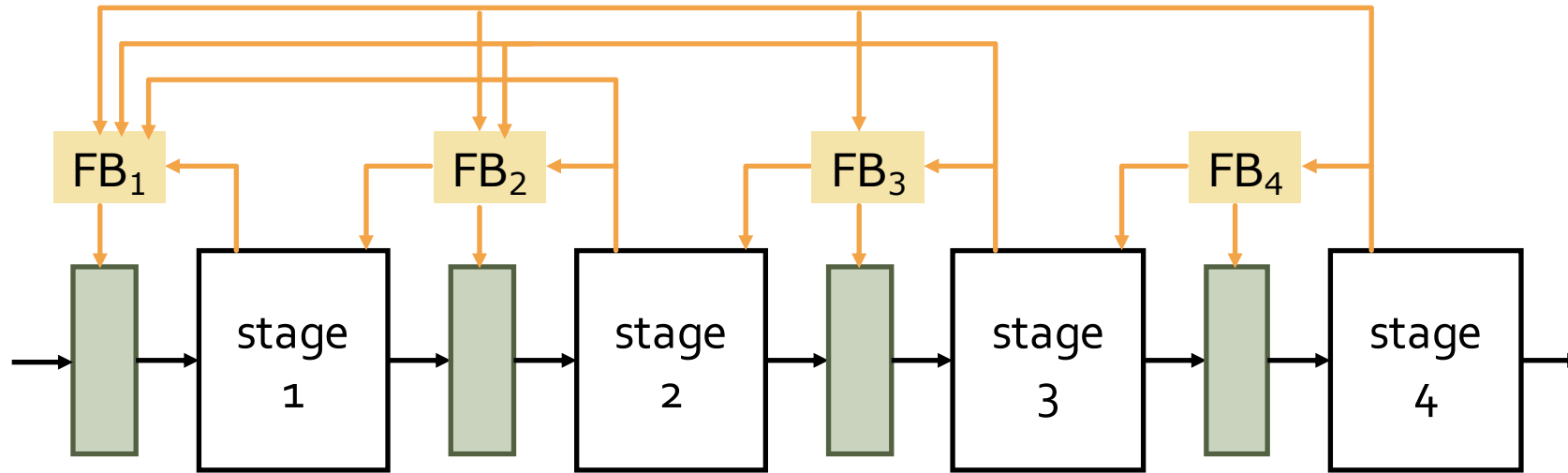
...

$x1$ is stale. Oops!

Overview of Data Hazards

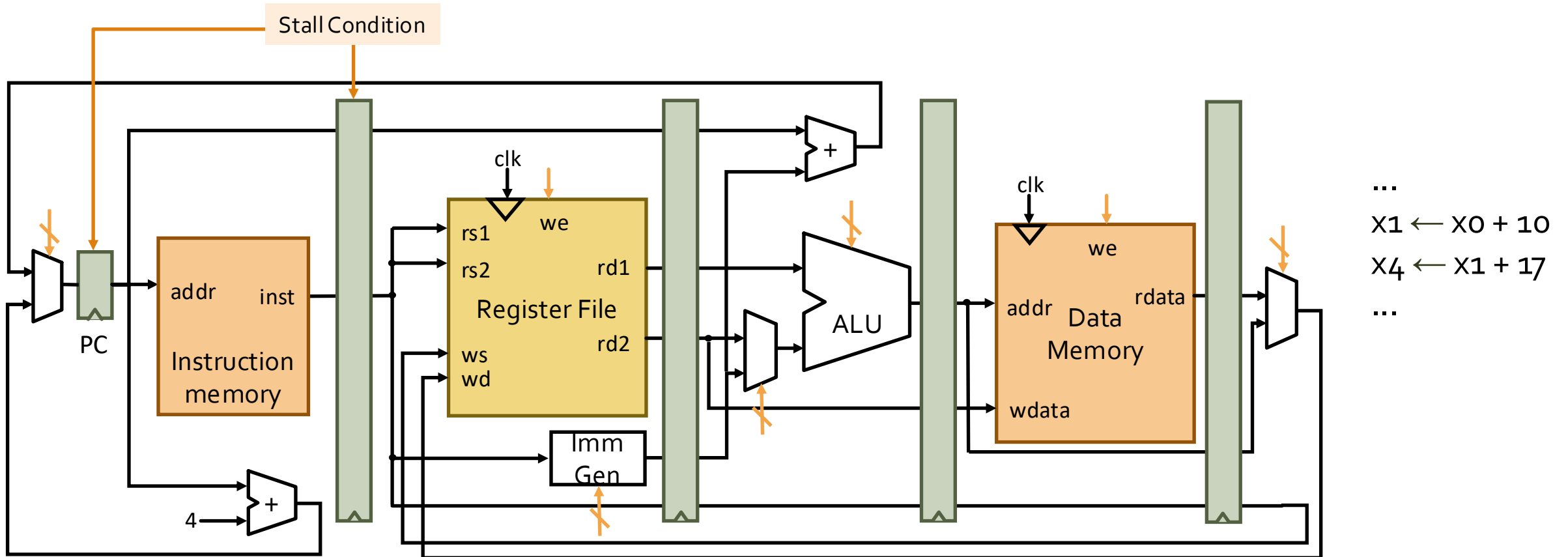
- Data hazard occurs when one instruction depends on a data value produced by a preceding instruction still in the pipeline
- Approaches to resolving data hazards
 - **Schedule:** Programmer explicitly avoids scheduling instructions that would create data hazards
 - **Stall:** Hardware includes control logic that freezes earlier stages until preceding instruction has finished producing data value
 - **Bypass:** Hardware datapath allows values to be sent to an earlier stage before preceding instruction has left the pipeline
 - **Speculate:** Guess that there is not a problem, if incorrect kill speculative instruction and restart

Feedback to Resolve Hazards

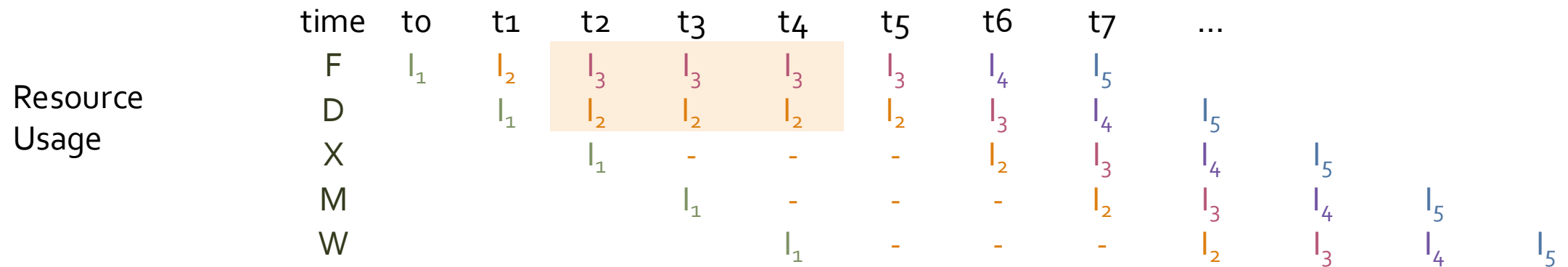
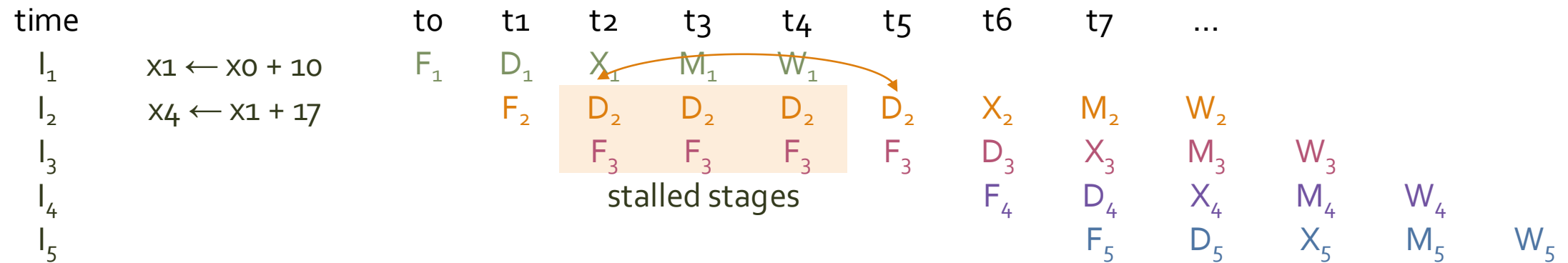


- Later stages provide dependence information to earlier stages which can **stall (or kill)** instructions
- Controlling a pipeline in this manner works provided the instruction at stage $i+1$ can complete without any interaction from instructions in stages 1 to i (otherwise deadlock)

Resolving Data Hazards with Stalls (Interlocks)

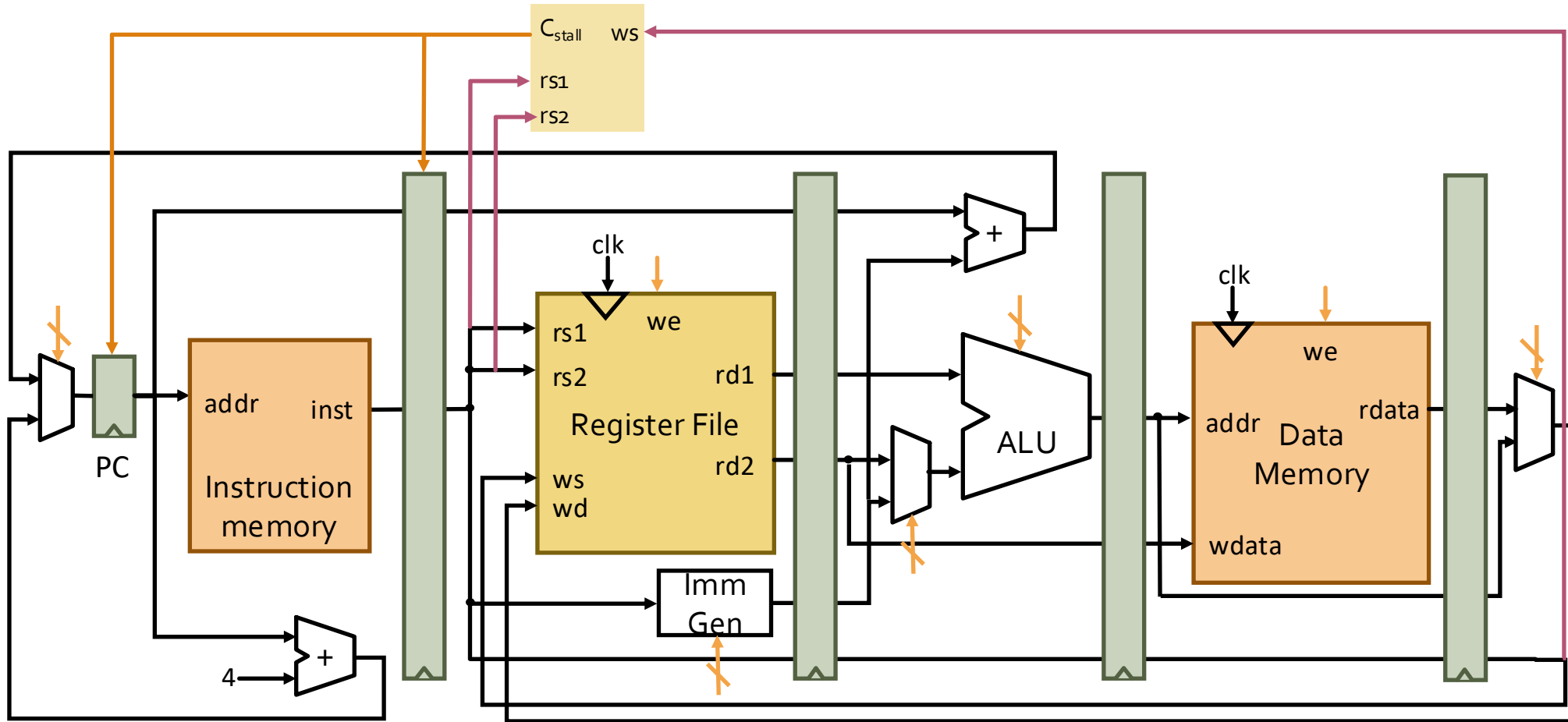


Stalled Stages and Pipeline Bubbles



- $\Rightarrow$ pipeline bubble

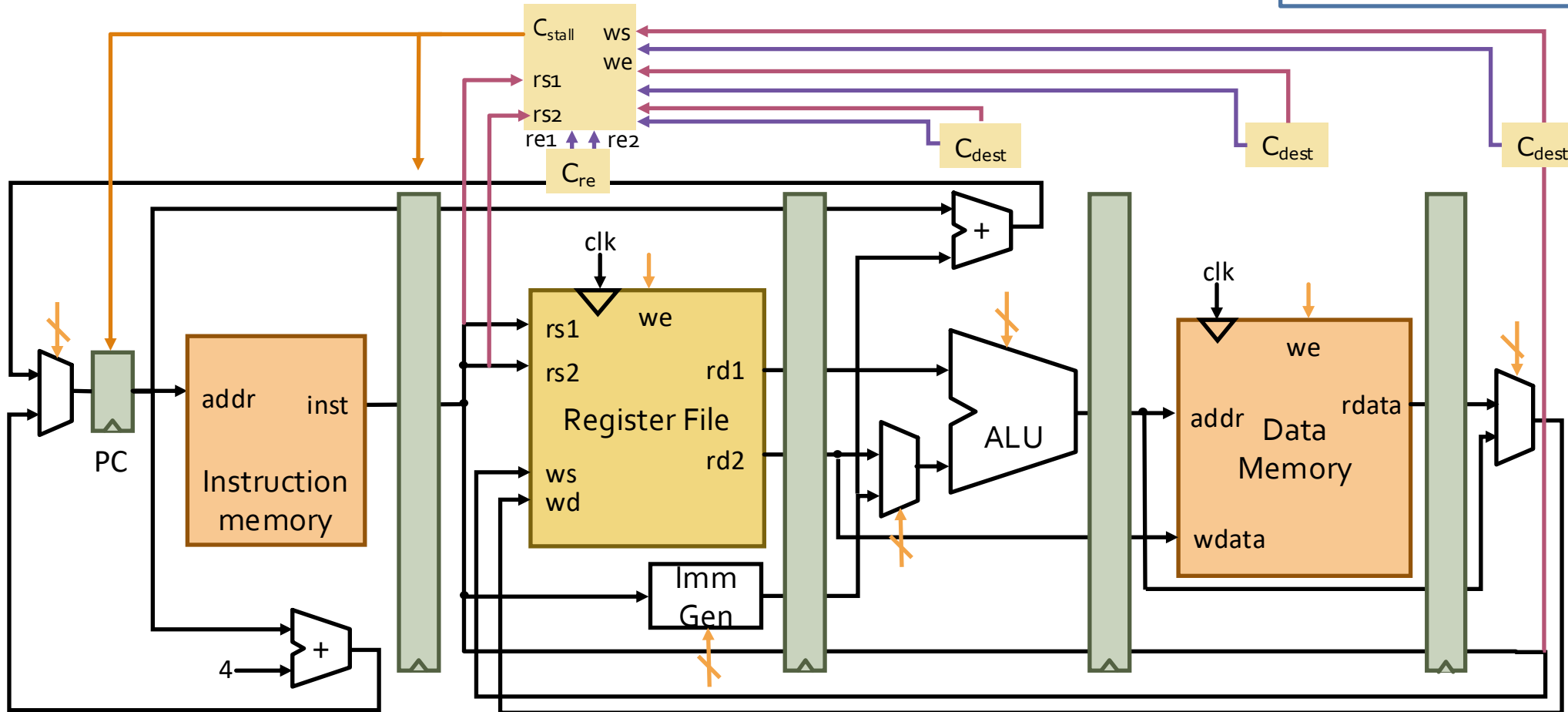
Stall Control Logic



Compare the source registers of the instruction in the decode stage with the destination register of the **uncommitted** instructions.

Stall Control Logic

we: write enable, 1-bit on/off
ws: write select, 5-bit register number
re: read enable, 1-bit on/off
rs: read select, 5-bit register number



Should we always stall if the
rs field matches some rd?

not every instruction writes a register → we
not every instruction reads a register → re

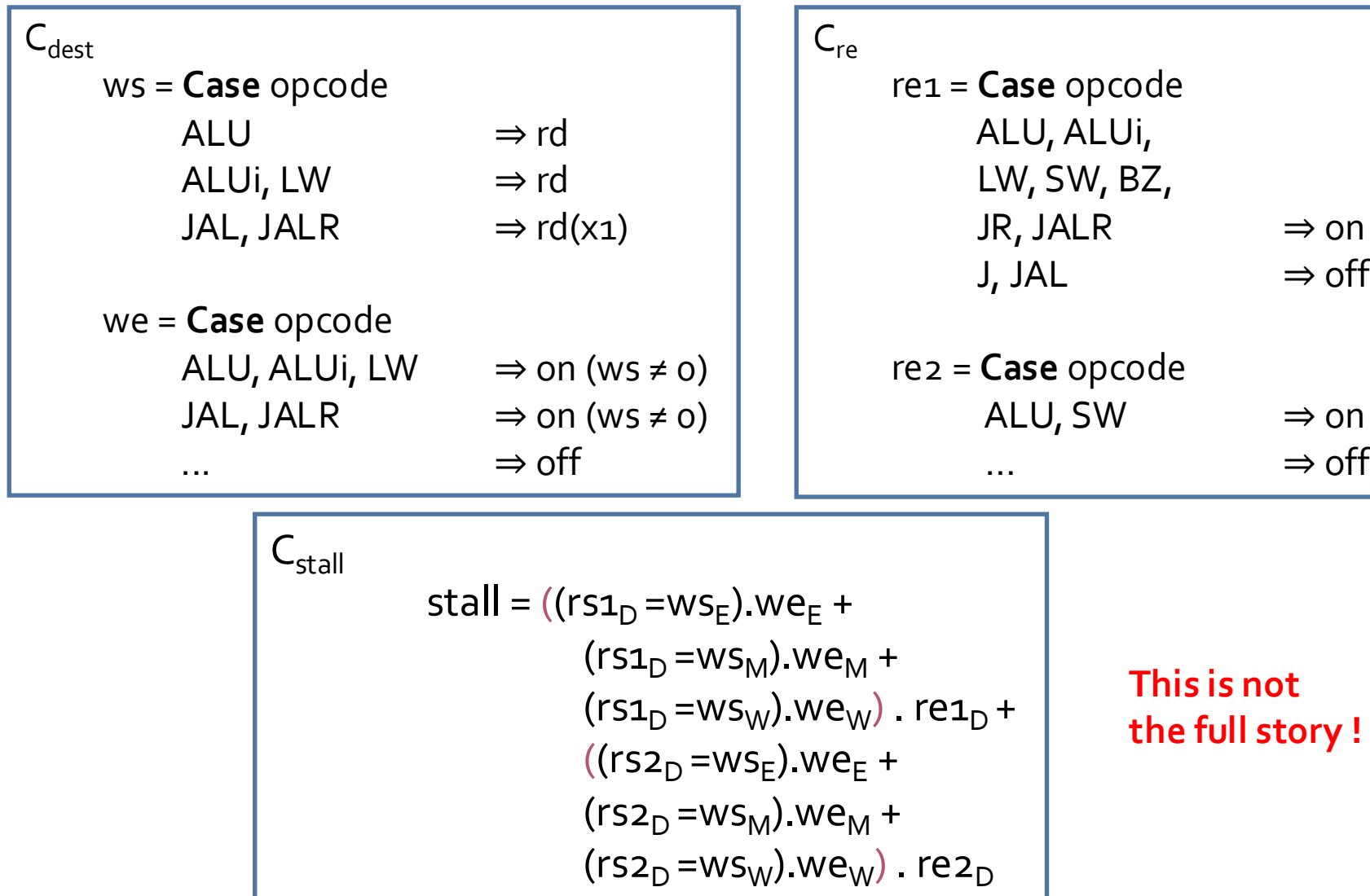
Source & Destination Registers

Name (Field Size)	Field						Comments
	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	
R-type	funct7	rs2	rs1	funct3	rd	opcode	Arithmetic instruction format
I-type	immediate[11:0]		rs1	funct3	rd	opcode	Loads & immediate arithmetic
S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode	Stores
SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode	Conditional branch format
UJ-type	immediate[20,10:1,11,19:12]				rd	opcode	Unconditional jump format
U-type	immediate[31:12]				rd	opcode	Upper immediate format

And more!

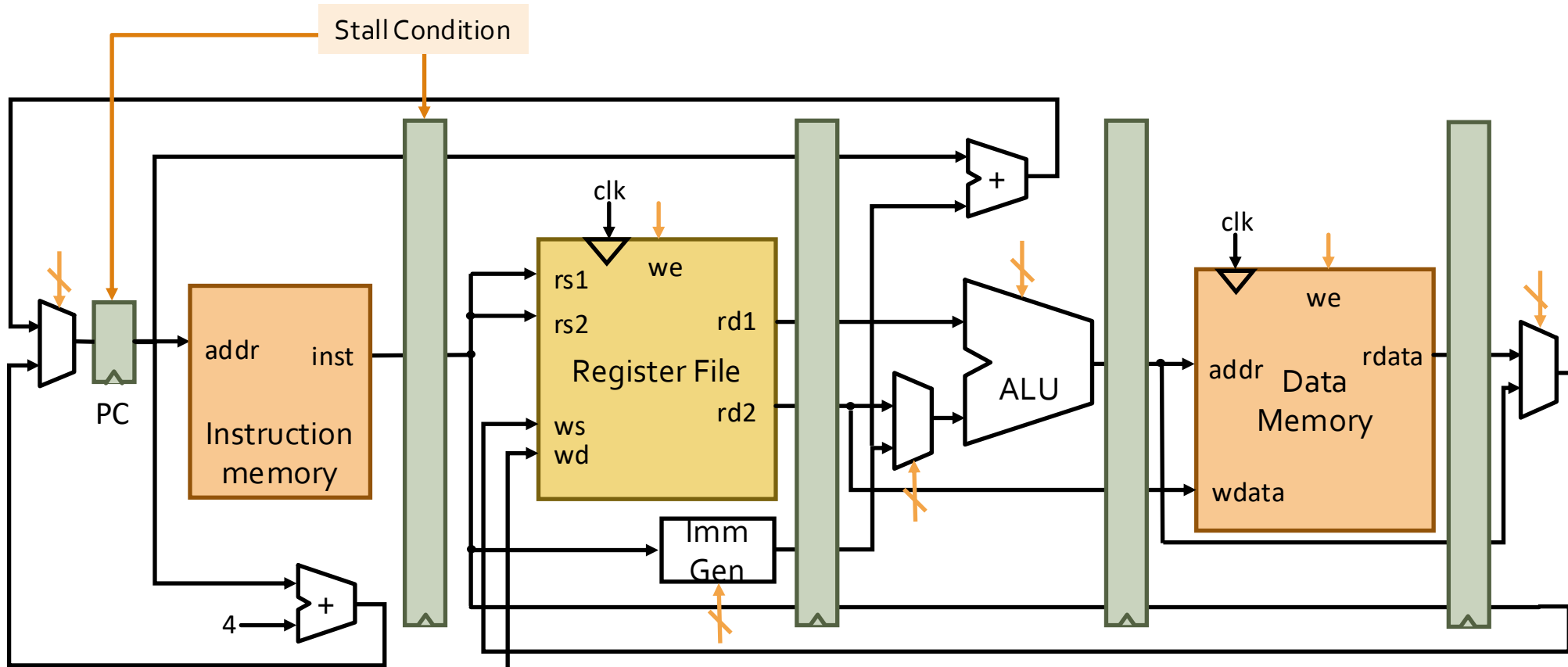
		Source	Destination
ALU	$rd \leftarrow (rs1) \text{ func } (rs2)$	rs1, rs2	rd
ALUi	$rd \leftarrow (rs1) \text{ func immediate}$	rs1	rd
LW	$rd \leftarrow M[(rs1) + \text{immediate}]$	rs1	rd
SW	$M[(rs1) + \text{immediate}] \leftarrow (rs2)$	rs1, rs2	
JAL	$rd \leftarrow (PC+4), PC \leftarrow (PC) + \text{immediate}$		rd
BEQ	If (rs1) == (rs2) then $PC \leftarrow (PC) + \text{immediate}$ else $PC \leftarrow (PC) + 4$	rs1, rs2	

Deriving the Stall Signal



Hazards due to Loads and Stores

What if $x1+7 = x3+5$?



Is there any possible data hazard in this instruction sequence?

...
 $M[x1+7] \leftarrow x2$
 $x4 \leftarrow M[x3+5]$
...

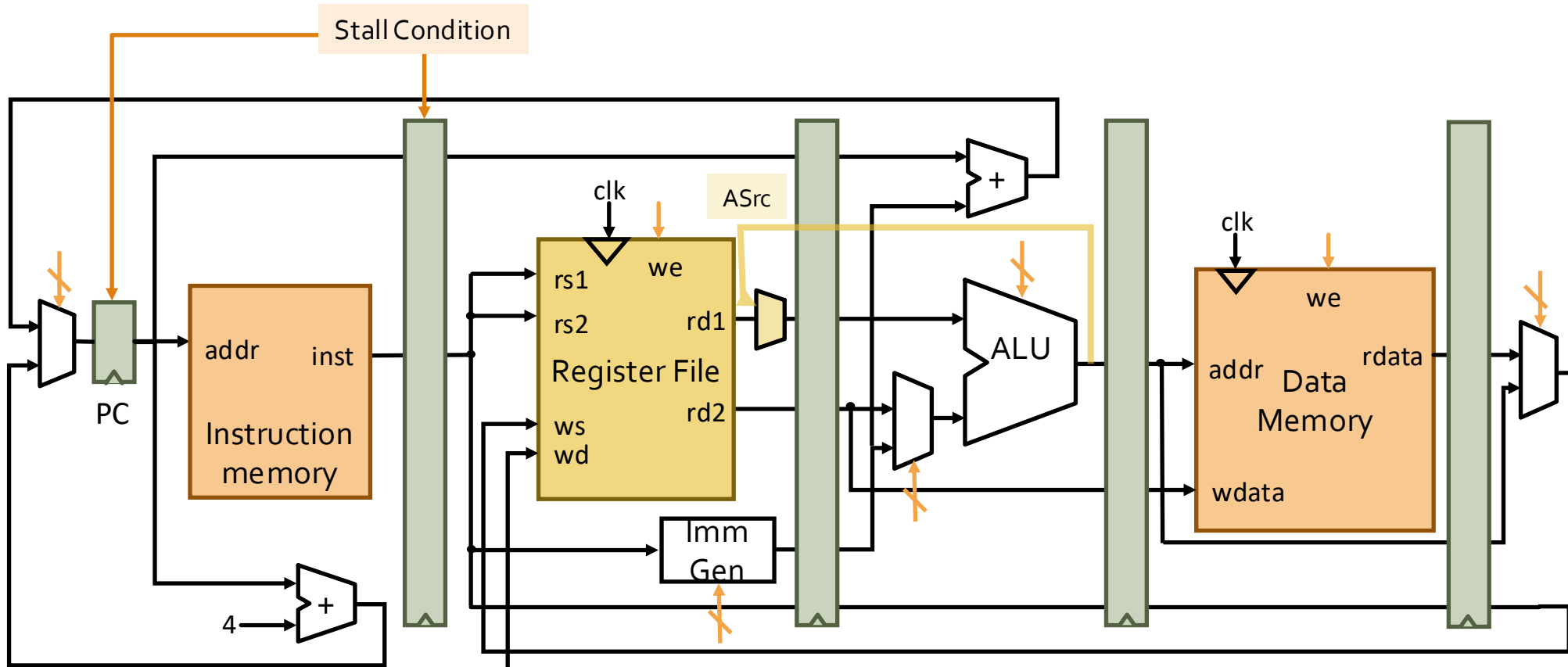
Load & Store Hazards

- Example instruction sequence
 - $M[x1+7] \leftarrow x2$
 - $x4 \leftarrow M[x3+5]$
- What if $x1+7 == x3+5$?
 - Writing and reading to/from the same address
 - Hazard is avoided because our memory system completes writes in a **single cycle**
 - More realistic memory system will require more careful handling of data hazards due to loads and stores

Overview of Data Hazards

- Data hazard occurs when one instruction depends on a data value produced by a preceding instruction still in the pipeline
- Approaches to resolving data hazards
 - **Schedule:** Programmer explicitly avoids scheduling instructions that would create data hazards
 - **Stall:** Hardware includes control logic that freezes earlier stages until preceding instruction has finished producing data value
 - **Bypass:** Hardware datapath allows values to be sent to an earlier stage before preceding instruction has left the pipeline
 - **Speculate:** Guess that there is not a problem, if incorrect kill speculative instruction and restart

Adding Bypassing to the Datapath



When does **this** bypass help?

$x1 \leftarrow x0 + 10$

$x4 \leftarrow x1 + 17$

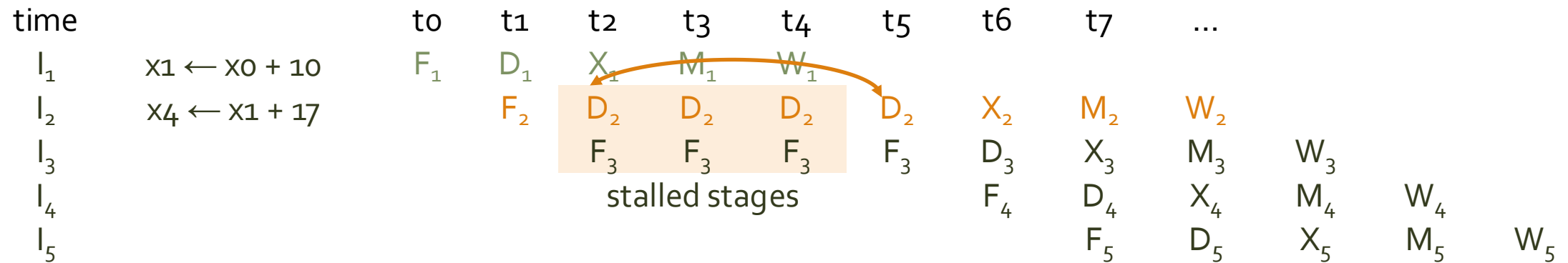
$x1 \leftarrow M[x0+10]$

$x4 \leftarrow x1 + 17$

JAL x1, 500

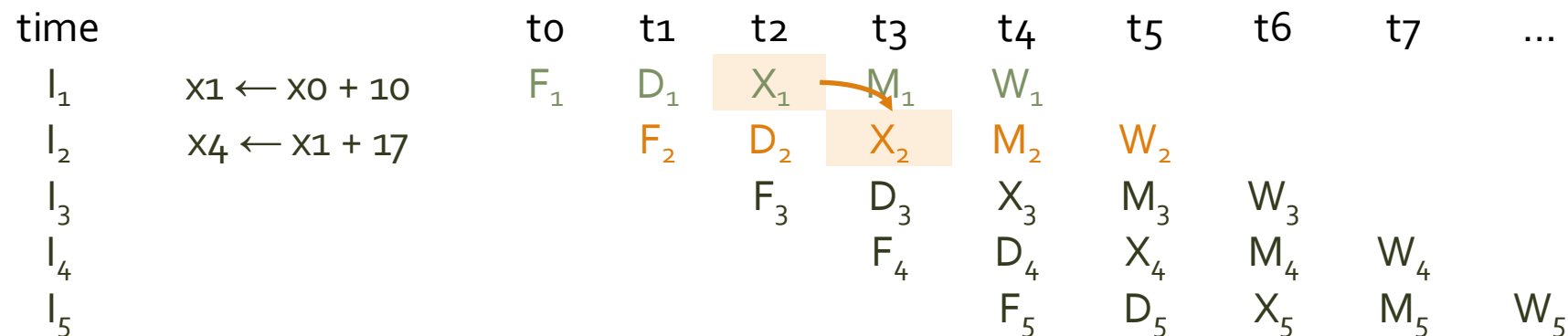
$x4 \leftarrow x1 + 17$

Deriving the Bypass Signal



Each **stall** or **kill** introduces a bubble in the pipeline $\Rightarrow CPI > 1$

A new datapath, i.e., a **bypass**, can get the data from the output of the ALU to its input



The Bypass Signal

Deriving it from the Stall Signal

$$\text{stall} = (\cancel{((rs1_D = ws_E).we_E)} + (rs1_D = ws_M).we_M + (rs1_D = ws_W).we_W).re1_D \\ + ((rs2_D = ws_E).we_E + (rs2_D = ws_M).we_M + (rs2_D = ws_W).we_W).re2_D)$$

ws = **Case** opcode

ALU $\Rightarrow$ rd

ALUi, LW $\Rightarrow$ rd

JAL, JALR $\Rightarrow$ rd

we = **Case** opcode

ALU, ALUi, LW $\Rightarrow$ on (ws $\neq$ o)

JAL, JALR $\Rightarrow$ on (ws $\neq$ o)

... $\Rightarrow$ off

$$\text{ASrc} = (rs1_D = ws_E).we_E.re1_D$$

Is this correct?

No! because only ALU and ALUi instructions can benefit from this bypass. Split we_E into two components: we-bypass, we-stall.

Bypass and Stall Signals

Split we_E into two components: we-bypass, we-stall

$we_bypass_E = \text{Case opcode}$

ALU, ALUi $\Rightarrow$ on ($ws \neq 0$)

... $\Rightarrow$ off

$we_stall_E = \text{Case opcode}$

LW $\Rightarrow$ on ($ws \neq 0$)

JAL, JALR $\Rightarrow$ on ($ws \neq 0$)

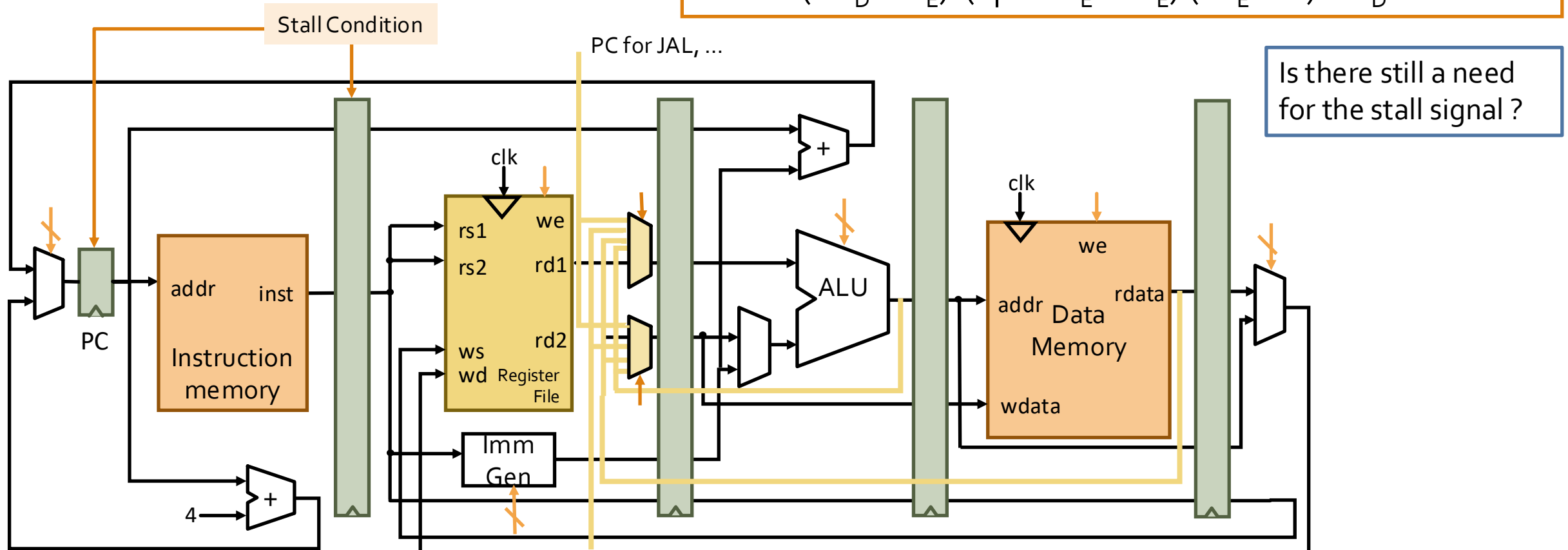
... $\Rightarrow$ off

$ASrc = (rs1_D = ws_E).we_bypass_E.re1_D$

$stall = (((rs1_D = ws_E).we_stall_E + (rs1_D = ws_M).we_M + (rs1_D = ws_W).we_W).re1_D$
 $+ ((rs2_D = ws_E).we_E + (rs2_D = ws_M).we_M + (rs2_D = ws_W).we_W).re2_D)$

Fully Bypassed Datapath

$$\text{stall} = (\text{rs1}_D = \text{ws}_E) \cdot (\text{opcode}_E = \text{LW}_E) \cdot (\text{ws}_E \neq 0) \cdot \text{re1}_D \\ + (\text{rs2}_D = \text{ws}_E) \cdot (\text{opcode}_E = \text{LW}_E) \cdot (\text{ws}_E \neq 0) \cdot \text{re2}_D$$



Overview of Data Hazards

- Data hazard occurs when one instruction depends on a data value produced by a preceding instruction still in the pipeline
- Approaches to resolving data hazards
 - **Schedule:** Programmer explicitly avoids scheduling instructions that would create data hazards
 - **Stall:** Hardware includes control logic that freezes earlier stages until preceding instruction has finished producing data value
 - **Bypass:** Hardware datapath allows values to be sent to an earlier stage before preceding instruction has left the pipeline
 - **Speculate:** Guess that there is not a problem, if incorrect kill speculative instruction and restart

Agenda

- ISA Review
- Pipeline Review
 - Pipelining Basics
 - Hazards
 - Structural Hazards
 - Data Hazards
 - Control Hazards

Control Hazards

- What do we need to calculate next PC?
 - For Jumps
 - Opcode, PC and offset
 - For Jump Register
 - Opcode, Register value, and PC
 - For Conditional Branches
 - Opcode, Register (for condition), PC and offset
 - For all other instructions
 - Opcode and PC
 - have to know it's not one of above

Opcode Decoding Bubble

Speculate next address
is PC+4

time		t0	t1	t2	t3	t4	t5	t6	t7	...	IS P	
I ₁	x1 ← x0 + 10	F ₁	D ₁	X ₁	M ₁	W ₁						
I ₂	x3 ← x2 + 17		F ₂	F ₂	D ₂	X ₂	M ₂	W ₂				
I ₃					F ₃	F ₃	D ₃	X ₃	M ₃	W ₃		
I ₄							F ₄	F ₄	D ₄	X ₄	M ₄	W ₄

	time	t0	t1	t2	t3	t4	t5	t6	t7	...		
Resource Usage	F	I ₁	-	I ₂	-	I ₃	-	I ₄	-			
	D		I ₁	-	I ₂	-	I ₃	-	I ₄	-		
	X			I ₁	-	I ₂	-	I ₃	-	I ₄	-	
	M				I ₁	-	I ₂	-	I ₃	-	I ₄	-
	W					I ₁	-	I ₂	-	I ₃	-	I ₄

CPI = 2!

- ⇒ pipeline bubble

Branch Pipeline Diagrams (resolved in execute stage)

time		to	t1	t2	t3	t4	t5	t6	t7	...
I ₁	096:ADD	F ₁	D ₁	X ₁	M ₁	W ₁				
I ₂	100: BEQZ + 200		F ₂	D ₂	X ₂	M ₂	W ₂			
I ₃	104: ADD			F ₃	D ₃	-	-	-		
I ₄	108:				F ₄	-	-	-	-	
I ₅	300: ADD					F ₅	D ₅	X ₅	M ₅	W ₅

	time	to	t1	t2	t3	t4	t5	t6	t7	...
Resource Usage	IF	I ₁	I ₂	I ₃	I ₄	I ₅				
	ID		I ₁	I ₂	I ₃	-	I ₅			
	EX			I ₁	I ₂	-	-	I ₅		
	MA				I ₁	I ₂	-	-	I ₅	
	WB					I ₁	I ₂	-	-	I ₅

- ⇒ pipeline bubble

Reducing Branch Penalty

- One pipeline bubble can be removed if an extra comparator is used in the Decode stage
 - But might elongate cycle time

Branch Delay Slots (expose control hazard to software)

- Change the ISA semantics so that the instruction that follows a jump or branch is always executed
 - Gives compiler the flexibility to put in a useful instruction where normally a pipeline bubble would have resulted

I_1	096:	ADD	
I_2	100:	BEQZ + 200	
I_3	104:	ADD	← Delay Slot instruction executed regardless of branch outcome
I_4	304:	ADD	

- Other techniques include more advanced branch prediction, which can dramatically reduce the branch penalty... (to come later)

Branch Pipeline Diagrams (branch delay slot)

time		t0	t1	t2	t3	t4	t5	t6	t7	...
I ₁	096: ADD	F ₁	D ₁	X ₁	M ₁	W ₁				
I ₂	100: BEQZ + 200		F ₂	D ₂	X ₂	M ₂	W ₂			
I ₃	104: ADD			F ₃	D ₃	X ₃	M ₃	W ₃		
I ₄	300: ADD				F ₄	D ₄	EX ₄	M ₄	W ₄	

	time	t0	t1	t2	t3	t4	t5	t6	t7	...
Resource Usage	IF	I ₁	I ₂	I ₃	I ₄					
	ID		I ₁	I ₂	I ₃	I ₄				
	EX			I ₁	I ₂	I ₃	I ₄			
	MA				I ₁	I ₂	I ₃	I ₄		
	WB					I ₁	I ₂	I ₃	I ₄	

Why an Instruction may not be dispatched every cycle ($CPI > 1$)?

- Full bypassing may be too expensive to implement
 - Typically, all frequently used paths are provided
 - Some infrequently used bypass paths may increase cycle time and counteract the benefit of reducing CPI
- Loads have two cycle latency
 - Instruction after load cannot use load result
- Jumps/Conditional branches may cause bubbles
 - Kill following instruction(s) if no delay slots

Machines with software-visible delay slots may execute significant number of NOP instructions inserted by the compiler. NOPs not counted in useful CPI (alternatively, increase instructions/program)

Other Control Hazards

- Exceptions
- Interrupts

More on this later in the course!

Recap

- ISA Review
- Pipeline Review
 - Pipelining Basics
 - Hazards
 - Structural Hazards
 - Data Hazards
 - Control Hazards

Acknowledgements

- These slides contain material developed and copyright by:
 - Arvind (MIT)
 - Krste Asanovic (MIT/UCB)
 - Joel Emer (Intel/MIT)
 - James Hoe (CMU)
 - John Kubiatowicz (UCB)
 - David Patterson (UCB)
 - Christopher Batten (Cornell)
 - David Wentzlaff (Princeton)
- MIT material derived from course 6.823
- UCB material derived from course CS252
- Cornell material derived from course ECE 4750
- Princeton material derived from course ECE 475